



Linux for Beginners

A Practical Guide

Chapter 10 of 12

Disk Management & the Filesystem in Depth

Contents

10.1	How Linux Represents Disks and Partitions
10.2	Viewing Disks and Partitions: lsblk and fdisk -l
10.3	Creating Partitions with fdisk
10.4	Formatting Partitions with mkfs
10.5	Mounting and Unmounting: mount and umount
10.6	Persistent Mounts: /etc/fstab
10.7	Verifying Disk Usage and Mounts: df and lsblk
10.8	Logical Volume Management (LVM) Basics
10.9	Symbolic Links vs Hard Links: ln -s and ln
10.10	Inodes: How Linux Really Stores Files
10.11	Finding Large Files and Disk Hogs: du
10.12	Swap Space Management
10.13	Practical Disk Management Scenarios
10.14	Essential Commands: Quick Reference
10.15	Chapter Summary

Chapter 10

Disk Management & the Filesystem in Depth

Every file you have ever created, every package you have ever installed, and every log line Linux has ever written lives on a physical or virtual disk somewhere — and Linux has a very specific, very learnable way of representing that disk, carving it into partitions, formatting it with a filesystem, and making it appear as ordinary folders you can *cd* into. This chapter pulls back the curtain on that machinery: how devices are named, how partitions and filesystems are created and mounted, how Logical Volume Management lets you resize storage without downtime, the real difference between a symbolic link and a hard link, what an inode actually is, and how to keep an eye on space before it runs out.

10.1 How Linux Represents Disks and Partitions

Linux exposes every disk as a **block device** — a special file under `/dev` that represents a chunk of addressable storage. Unlike regular files, you never open a block device with a text editor; the kernel and tools like `mkfs` and `mount` talk to it directly. Every disk gets its own device name, and every partition on that disk gets a number appended to it.

Device naming conventions

The naming scheme depends on the type of storage controller. Older SATA, SCSI, and USB drives use the `sd` prefix, while modern NVMe SSDs use the `nvme` prefix with a slightly different partition naming rule.

Storage Type	Whole-Disk Device	First Partition	Second Partition
SATA / SCSI / USB drive	<code>/dev/sda</code>	<code>/dev/sda1</code>	<code>/dev/sda2</code>
Second SATA drive	<code>/dev/sdb</code>	<code>/dev/sdb1</code>	<code>/dev/sdb2</code>
NVMe SSD	<code>/dev/nvme0n1</code>	<code>/dev/nvme0n1p1</code>	<code>/dev/nvme0n1p2</code>
Virtio disk (KVM/QEMU VMs)	<code>/dev/vda</code>	<code>/dev/vda1</code>	<code>/dev/vda2</code>
MMC / SD card	<code>/dev/mmcblk0</code>	<code>/dev/mmcblk0p1</code>	<code>/dev/mmcblk0p2</code>

Notice the extra `p` before the partition number on NVMe and MMC devices — it disambiguates the partition number from a device name that already ends in a digit (`nvme0n1`).

```
$ ls -l /dev/sd*
brw-rw---- 1 root disk 8,  0 Aug 12 09:14 /dev/sda
brw-rw---- 1 root disk 8,  1 Aug 12 09:14 /dev/sda1
brw-rw---- 1 root disk 8,  2 Aug 12 09:14 /dev/sda2
brw-rw---- 1 root disk 8, 16 Aug 12 09:14 /dev/sdb

# The leading 'b' means block device (as opposed to 'c' for character device)
```

TIP

You rarely need to memorize these rules by heart. Run `lsblk` (covered next) and it draws the whole disk-and-partition tree for you, including mount points.

MBR vs GPT: two ways to lay out a disk

Every disk needs a partition table describing where each partition begins and ends. The older **MBR** (Master Boot Record) scheme supports at most four primary partitions and disks up to 2 TB; the modern **GPT** (GUID Partition Table) scheme supports up to 128 partitions by default and disks far larger than 2 TB. New installations on modern hardware default to GPT.

10.2 Viewing Disks and Partitions: `lsblk` and `fdisk -l`

Before you change anything about a disk, look at it. `lsblk` ("list block devices") gives you a fast, readable tree of every disk, partition, and mount point on the system, and it needs no special privileges.

```
$ lsblk
NAME            MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda              8:0    0   40G  0 disk
├─sda1           8:1    0    1G  0 part /boot
└─sda2           8:2    0   39G  0 part /
sdb              8:16   0  100G  0 disk
└─sdb1           8:17   0  100G  0 part /mnt/data
nvme0n1         259:0    0   1.8T  0 disk
└─nvme0n1p1     259:1    0   1.8T  0 part /home
```

Add `-f` to see filesystem type and UUID, which is invaluable when you are about to write an `fstab` entry:

```

$ lsblk -f
NAME            FSTYPE LABEL  UUID                                  MOUNTPOINTS
sda
|-sda1          ext4      3a1c9f2e-1b44-4e2a-9c2d-1122aabbccdd /boot
`-sda2          ext4      7f5e0d3c-88aa-4c11-9d21-33ffeeaabbcc /
sdb
`-sdb1          xfs       e4d1a9b0-22cc-4dd6-8a90-99bbccddeeff /mnt/data

```

For a much more detailed, partition-table-level view — including exact start and end sectors, partition types, and disk label format — use `fdisk -l`. This one needs `sudo` because it reads the raw partition table.

```

$ sudo fdisk -l /dev/sdb
Disk /dev/sdb: 100 GiB, 107374182400 bytes, 209715200 sectors
Disk model: Virtual Disk
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 4C6A8F2E-1234-4B77-9AAB-0123456789AB

Device      Start      End  Sectors Size Type
/dev/sdb1   2048 209715166 209713119 100G Linux filesystem

```

Tool	Needs sudo?	Best For
<code>lsblk</code>	No	Quick overview: what's attached, what's mounted, sizes
<code>lsblk -f</code>	No	Filesystem types and UUIDs, for <code>fstab</code> entries
<code>fdisk -l</code>	Yes	Exact partition boundaries, sector sizes, disklabel type
<code>blkid</code>	Yes (usually)	UUID and filesystem type for a single device

TIP

Run `lsblk` first, every time. It's the fastest way to get your bearings before touching a disk.

10.3 Creating Partitions with `fdisk`

To create a new partition, open the target disk (never a partition, the whole disk) in interactive mode with `fdisk`. Nothing is written to disk until you press `w`, so it's safe to explore.

```
$ sudo fdisk /dev/sdb
```

BASH

```
Welcome to fdisk (util-linux 2.39.3).
```

```
Changes will remain in memory only, until you decide to write them.
```

```
Command (m for help): n
```

```
Partition type
```

```
  p   primary (0 primary, 0 extended, 4 free)
```

```
  e   extended
```

```
Select (default p): p
```

```
Partition number (1-4, default 1): 1
```

```
First sector (2048-209715199, default 2048): [Enter]
```

```
Last sector (2048-209715199, default 209715199): +100G
```

```
Created a new partition 1 of type 'Linux filesystem' and of size 100 GiB.
```

```
Command (m for help): p
```

```
Device      Start      End      Sectors Size Type
/dev/sdb1    2048 209715199 209713152 100G Linux filesystem
```

```
Command (m for help): w
```

```
The partition table has been altered.
```

```
Calling ioctl() to re-read partition table.
```

```
Syncing disks.
```

Key	Action
m	Show the help menu of all commands
p	Print the current in-memory partition table
n	Create a new partition
d	Delete a partition
t	Change a partition's type code
w	Write the partition table to disk and exit
q	Quit without saving any changes
l	List all known partition type codes

WARNING

fdisk operates on the raw device. Running it against the wrong disk (for example /dev/sda, your boot disk, instead of /dev/sdb) and pressing w can destroy your operating system. Always confirm the target with lsblk first, and never fdisk a disk that is currently mounted and in use.

For disks larger than 2 TB or when you specifically need a GPT layout, parted is the more modern equivalent and supports the same concepts with a slightly different interactive syntax (`mklabel gpt`, `mkpart`).

10.4 Formatting Partitions with mkfs

A freshly partitioned block of storage has no filesystem yet — it's just raw space. `mkfs` ("make filesystem") writes the data structures a filesystem needs: the superblock, inode tables, and free-space bitmaps.

```
$ sudo mkfs.ext4 /dev/sdb1
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 26214144 4k blocks and 6553600 inodes
Filesystem UUID: 9c2a1e77-44bb-4a0d-9f21-aabbccddeeff
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736

Allocating group tables: done
Writing inode tables: done
Writing superblocks and filesystem accounting information: done

$ sudo mkfs.xfs /dev/sdc1
meta-data=/dev/sdc1      isize=512    agcount=4, agsize=6553600 blks
data           =         bsize=4096   blocks=26214400, imaxpct=25

$ sudo mkfs.vfat -F32 /dev/sdd1
mkfs.fat 4.2 (2021-01-31)
```

BASH

Filesystem	Command	Journaling?	Typical Use
ext4	<code>mkfs.ext4</code>	Yes	General-purpose Linux default, very mature
XFS	<code>mkfs.xfs</code>	Yes	Large files, high-throughput servers, RHEL default
Btrfs	<code>mkfs.btrfs</code>	Yes (CoW)	Snapshots, checksums, built-in volume management
FAT32	<code>mkfs.vfat -F32</code>	No	USB drives shared with Windows/macOS, 4GB file limit
NTFS	<code>mkfs.ntfs</code>	Yes	Drives that must stay readable/writable on Windows

WARNING

`mkfs` erases everything on the target partition. Double-check the device path with `lsblk` before running it — there is no confirmation prompt and no undo.

10.5 Mounting and Unmounting: mount and umount

A formatted partition is invisible to the rest of the filesystem tree until you **mount** it — attach it to an existing directory, called a *mount point*. From that moment, files inside the mount point directory are actually stored on the mounted device.

```
BASH
$ sudo mkdir -p /mnt/data
$ sudo mount /dev/sdb1 /mnt/data
$ mount | grep sdb1
/dev/sdb1 on /mnt/data type ext4 (rw,relatime)

$ df -h /mnt/data
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdb1        98G   24K   93G   1% /mnt/data

$ sudo umount /mnt/data
```

You can also unmount by mount point instead of device name, which is often easier to remember:

```
BASH
$ sudo umount /mnt/data
```

Mount Option	Meaning
ro	Mount read-only
rw	Mount read-write (default)
noexec	Prevent executing binaries from this filesystem
nosuid	Ignore setuid/setgid bits on this filesystem
nodev	Ignore device files on this filesystem
noatime	Don't update file access times — improves performance

If `umount` refuses with *"target is busy"*, something still has an open file or a shell session inside that mount point. Find the culprit with `lsof +f -- /mnt/data` or `fuser -vm /mnt/data` before forcing anything.

10.6 Persistent Mounts: /etc/fstab

A manual mount disappears on reboot. To have a partition mount automatically every time the system starts, add an entry to `/etc/fstab` ("filesystem table"). Each line has six whitespace-separated fields.

Field	Meaning
1. Device	UUID=... (preferred) or /dev/sdXN
2. Mount point	Directory the filesystem attaches to, e.g. /mnt/data
3. Type	Filesystem type: ext4, xfs, vfat, swap, ...
4. Options	Comma-separated mount options, e.g. defaults,noatime
5. Dump	Used by the legacy dump backup tool; 0 disables it
6. Pass	fsck order at boot: 1 for /, 2 for other local disks, 0 to skip

Device names like /dev/sdb1 can shift if you add or remove drives, so fstab entries should reference the partition's stable **UUID** instead. Get it with `blkid`:

```

$ sudo blkid /dev/sdb1
/dev/sdb1: UUID="e4d1a9b0-22cc-4dd6-8a90-99bbccddee" TYPE="ext4"

$ sudo nano /etc/fstab
# <device>          <mount point> <type> <options>    <dump> <pass>
UUID=e4d1a9b0-...-99bbccddee /mnt/data     ext4    defaults,noatime 0      2

```

WARNING

A typo in /etc/fstab can leave the system unable to boot into a normal login. Before rebooting, always test the file with 'sudo mount -a' — it mounts every fstab entry immediately and will report an error without touching your running session.

```

$ sudo mount -a
$ mount | grep /mnt/data

```

10.7 Verifying Disk Usage and Mounts: df and lsblk

`df` ("disk free") reports space usage per mounted filesystem — the single most common command for "am I about to run out of space?"

```

$ df -h
BASH
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2       39G   14G   23G   38% /
/dev/sda1       1.0G  180M   760M   20% /boot
/dev/sdb1       98G   24G   70G   26% /mnt/data
tmpfs           3.9G    0   3.9G    0% /dev/shm

$ df -hT /mnt/data
Filesystem      Type  Size  Used Avail Use% Mounted on
/dev/sdb1       ext4   98G   24G   70G   26% /mnt/data

$ df -i /
Filesystem      Inodes   IUsed   IFree IUse% Mounted on
/dev/sda2       2555904 412300 2143604   17% /

```

For a single mount point, `findmnt` gives a cleaner, more detailed answer than grepping `mount`:

```

$ findmnt /mnt/data
BASH
TARGET      SOURCE      FSTYPE OPTIONS
/mnt/data   /dev/sdb1   ext4     rw,relatime

```

TIP

`-h` on `df` and `du` means human-readable (G, M, K) instead of raw byte counts. Get in the habit of always adding it.

10.8 Logical Volume Management (LVM) Basics

Plain partitions are fixed in size the moment you create them. **LVM** adds a flexible layer on top of disks so you can resize, combine, and snapshot storage without unmounting anything. LVM has three building blocks:

Layer	Command to Create	What It Is
Physical Volume (PV)	<code>pvcreate</code>	A raw disk or partition registered for LVM use
Volume Group (VG)	<code>vgcreate</code>	A pool combining one or more PVs into shared free space
Logical Volume (LV)	<code>lvcreate</code>	A resizable "virtual partition" carved from a VG

```

# 1. Register the raw partition as a physical volume
$ sudo pvcreate /dev/sdc1
Physical volume "/dev/sdc1" successfully created.

# 2. Create a volume group from it
$ sudo vgcreate vgdata /dev/sdc1
Volume group "vgdata" successfully created

# 3. Carve out a 10GB logical volume
$ sudo lvcreate -L 10G -n lvweb vgdata
Logical volume "lvweb" created.

# 4. Format and mount it like any other block device
$ sudo mkfs.ext4 /dev/vgdata/lvweb
$ sudo mount /dev/vgdata/lvweb /mnt/web

```

The real payoff comes later. If `lvweb` starts filling up and the volume group still has free space, you can grow it online — while it's mounted and in use:

```

$ sudo lvextend -L +5G /dev/vgdata/lvweb
Size of logical volume vgdata/lvweb changed from 10.00 GiB to 15.00 GiB.

$ sudo resize2fs /dev/vgdata/lvweb
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/vgdata/lvweb is mounted on /mnt/web; on-line resizing required
The filesystem on /dev/vgdata/lvweb is now 3932160 (4k) blocks long.

```

Command	Purpose
<code>pvcreate / pvdisplay</code>	Create / inspect a physical volume
<code>vgcreate / vgdisplay</code>	Create / inspect a volume group
<code>vgextend</code>	Add another physical volume into an existing group
<code>lvcreate / lvdisplay</code>	Create / inspect a logical volume
<code>lvextend</code>	Grow a logical volume (pair with <code>resize2fs</code> or <code>xfs_growfs</code>)
<code>lvreduce</code>	Shrink a logical volume (ext4 only, and riskier — back up first)

TIP

`resize2fs` works for ext4; XFS volumes use `xfs_growfs` instead. XFS filesystems can only ever grow, never shrink.

10.9 Symbolic Links vs Hard Links: `ln -s` and `ln`

Linux offers two very different ways to make a file appear in more than one place. Understanding the difference prevents a lot of confusion later.

```
# Symbolic link (symlink) – a pointer to a path
$ ln -s /opt/myapp/config.yml ~/config.yml
$ ls -l ~/config.yml
lrwxrwxrwx 1 ashik ashik 24 Aug 20 10:02 config.yml -> /opt/myapp/config.yml

# Hard link – a second name for the same inode
$ ln /var/data/report.csv /var/data/report-backup.csv
$ ls -li /var/data/report.csv /var/data/report-backup.csv
812004 -rw-r--r-- 2 ashik ashik 4096 Aug 20 10:02 report-backup.csv
812004 -rw-r--r-- 2 ashik ashik 4096 Aug 20 10:02 report.csv
```

Note the matching inode number 812004 on both hard-linked files, and the link count of 2 in the third column.

Property	Symbolic Link	Hard Link
Points to	A path (string)	The same inode (data) directly
Crosses filesystems?	Yes	No — must be on the same filesystem
Can link a directory?	Yes	No (not permitted for regular users)
If original is deleted	Becomes a "broken" link	File still exists; data is untouched
Looks like in <code>ls -l</code>	Starts with <code>l</code> , shows <code>-></code> target	Looks like a normal file

```
$ readlink -f ~/config.yml
/opt/myapp/config.yml
```

TIP

`readlink -f` resolves a symlink (even a chain of them) all the way down to the real, final path.

10.10 Inodes: How Linux Really Stores Files

Every file on a Linux filesystem is represented by an **inode** — a data structure holding all of a file's metadata: permissions, owner, size, timestamps, and pointers to the actual data blocks on disk. Crucially, the inode does **not** store the filename. The filename lives in the directory entry, which is just a mapping from a name to an inode number.

This is exactly why a hard link works: two directory entries with different names can point at the same inode, and the file data isn't removed until every hard link to it is gone.

```
BASH
$ stat /etc/hostname
  File: /etc/hostname
  Size: 8          Blocks: 8          IO Block: 4096   regular file
Device: 8,2      Inode: 131590       Links: 1
Access: (0644/-rw-r--r--)  Uid: (  0/   root)   Gid: (  0/   root)
Modify: 2026-07-14 09:11:02.000000000 +0530

$ ls -li /etc/hostname
131590 /etc/hostname
```

Stored in the Inode	NOT Stored in the Inode
Permissions and ownership (UID/GID)	The filename
File size and timestamps	The directory it "lives in"
Pointers to the data blocks on disk	
Link count (how many names point to it)	

The inode trap: out of space with space to spare

Every filesystem is created with a fixed number of inodes, decided at `mkfs`'s time. If a workload creates millions of tiny files (session caches, mail queues, build artifacts), you can exhaust every available inode while `df -h` still shows plenty of free bytes. The symptom is a confusing *"No space left on device"* error. The fix is to check inode usage specifically:

```
BASH
$ df -li /var
Filesystem      Inodes    IUsed    IFree IUse% Mounted on
/dev/sda2       2555904  2555899        5  100% /
```

10.11 Finding Large Files and Disk Hogs: `du`

While `df` reports space at the filesystem level, `du` ("disk usage") reports space consumed by specific files and directories — exactly what you need when a disk is full and you must find out

why.

```
$ du -sh /var/log
1.2G    /var/log

$ du -h --max-depth=1 /var
1.2G    /var/log
340M    /var/cache
88M     /var/lib
1.6G    /var

$ du -ah /var/log | sort -rh | head -5
820M    /var/log/journal
310M    /var/log/nginx/access.log
45M     /var/log/nginx/error.log
12M     /var/log/syslog
6.1M    /var/log/auth.log
```

BASH

Flag	Meaning
-s	Summarize: show one total per argument, not every subdirectory
-h	Human-readable sizes (K, M, G)
-a	Include individual files, not just directories
--max-depth=N	Only descend N levels before summarizing
-x	Stay on one filesystem — don't cross into other mounted disks

TIP

For a genuinely interactive way to explore disk usage, install `ncdu` (`sudo apt install ncdu`) — it turns 'du' into a navigable, ranked terminal UI.

One subtlety: `du` and `df` can disagree. If a process still has a large deleted file open, the space stays allocated (so `df` shows it as used) even though nothing on disk shows up in a directory listing (so `du` can't find it). `ls -l +L` lists open files with a link count of zero — exactly this situation.

10.12 Swap Space Management

Swap is disk space the kernel uses as overflow when physical RAM is full, moving inactive memory pages out to disk to free up RAM for active processes. It won't make a system fast, but it can be the difference between a process getting killed and a system that survives a memory spike.

```

$ free -h
              total        used        free      shared  buff/cache   available
Mem:           3.8Gi        1.2Gi        800Mi        45Mi         1.8Gi         2.4Gi
Swap:          2.0Gi          0B         2.0Gi

$ swapon --show
NAME        TYPE      SIZE USED PRIO
/swapfile   file      2G      0B  -2
```

Creating a swap file

On a VPS or cloud instance without a dedicated swap partition, a swap file is the simplest option:

```

$ sudo fallocate -l 2G /swapfile
$ sudo chmod 600 /swapfile
$ sudo mkswap /swapfile
Setting up swapspace version 1, size = 2 GiB (2147479552 bytes)
$ sudo swapon /swapfile
$ free -h
Swap:           2.0Gi          0B         2.0Gi

# Make it permanent across reboots
$ echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

WARNING
chmod 600 on the swap file is not optional — swap can contain sensitive data swapped out of RAM, and a world-readable swap file is a real information leak.

Command	Purpose
free -h	See current RAM and swap usage
swapon --show	List active swap devices/files
mkswap /path	Initialize a file or partition as swap space
swapon /path	Activate a swap file or partition
swapoff /path	Deactivate swap (moves data back into RAM first)

TIP

The `vm.swappiness` sysctl (0-100) controls how eagerly the kernel swaps: lower values prefer keeping data in RAM, higher values swap more readily. Desktops often lower it; servers with tight RAM sometimes raise it.

10.13 Practical Disk Management Scenarios

Scenario 1: Adding a brand-new disk to a server

1. Run `lsblk` to confirm the new disk's device name (e.g. it appears as `/dev/sdc`, unpartitioned).
2. Partition it: `sudo fdisk /dev/sdc`, then `n`, accept the defaults, then `w`.
3. Format it: `sudo mkfs.ext4 /dev/sdc1`.
4. Create a mount point and mount it: `sudo mkdir /mnt/newdisk`; `sudo mount /dev/sdc1 /mnt/newdisk`.
5. Get its UUID with `blkid` and add a line to `/etc/fstab` so it survives a reboot.
6. Verify with `sudo mount -a` and `df -h` before rebooting.

Scenario 2: "No space left on device" but `df` shows free space

1. Run `df -h` — if it genuinely shows free space, the disk isn't the problem in the way you think.
2. Check inodes next: `df -i`. A 100% IUse column means you're out of inodes, not bytes.
3. If inodes are fine, suspect deleted-but-open files: `lsdf +L1` or `lsdf | grep deleted`.
4. Restarting the service holding the deleted file open (or having it reopen its log) releases the space immediately.

Scenario 3: Fixing OOM kills on a small VPS

1. Confirm there's no existing swap: `free -h` shows Swap: 0B.
2. Create a swap file sized to roughly your RAM (or half of it, for larger instances): `fallocate -l 2G /swapfile`.
3. `chmod 600` it, `mkswap` it, then `swapon` it.
4. Add the `fstab` entry so it survives reboot, then re-check with `free -h`.

Scenario 4: "What exactly is mounted, and from where?"

1. `findmnt` with no arguments prints the entire mount tree at a glance.
2. `findmnt /path/in/question` resolves precisely which device backs a specific directory.
3. `cat /proc/mounts` shows the kernel's live, authoritative view — useful when `mount` and `fstab` seem to disagree.

4. `mount | grep keyword` is the quick, classic one-liner for a fast lookup.

10.14 Essential Commands: Quick Reference

Command	What It Does	Common Usage
<code>lsblk</code>	Lists block devices in a tree	<code>lsblk / lsblk -f</code>
<code>fdisk -l</code>	Lists partition tables in detail	<code>sudo fdisk -l /dev/sdb</code>
<code>fdisk /dev/sdX</code>	Interactively partitions a disk	<code>sudo fdisk /dev/sdb</code>
<code>parted</code>	Modern partitioning tool, handles GPT and 2TB+ disks	<code>sudo parted /dev/sdb</code>
<code>mkfs.ext4</code>	Formats a partition as ext4	<code>sudo mkfs.ext4 /dev/sdb1</code>
<code>mkfs.xfs</code>	Formats a partition as XFS	<code>sudo mkfs.xfs /dev/sdc1</code>
<code>mount</code>	Attaches a filesystem to a directory	<code>sudo mount /dev/sdb1 /mnt/data</code>
<code>umount</code>	Detaches a mounted filesystem	<code>sudo umount /mnt/data</code>
<code>blkid</code>	Shows a device's UUID and filesystem type	<code>sudo blkid /dev/sdb1</code>
<code>findmnt</code>	Shows the current mount tree or a specific mount	<code>findmnt /mnt/data</code>
<code>df -h</code>	Shows space usage per mounted filesystem	<code>df -h</code>
<code>df -i</code>	Shows inode usage per mounted filesystem	<code>df -i /</code>
<code>du -sh</code>	Shows total size of a directory	<code>du -sh /var/log</code>
<code>pvcreate</code>	Registers a device as an LVM physical volume	<code>sudo pvcreate /dev/sdc1</code>
<code>vgcreate</code>	Creates an LVM volume group	<code>sudo vgcreate vgdata /dev/sdc1</code>
<code>lvcreate</code>	Creates an LVM logical volume	<code>sudo lvcreate -L 10G -n lvweb vgdata</code>
<code>lvextend</code>	Grows an LVM logical volume online	<code>sudo lvextend -L +5G /dev/vgdata/lvweb</code>
<code>resize2fs</code>	Resizes an ext4 filesystem to match its volume	<code>sudo resize2fs /dev/vgdata/lvweb</code>
<code>ln -s</code>	Creates a symbolic link	<code>ln -s /path/target linkname</code>
<code>ln</code>	Creates a hard link	<code>ln file.txt file-backup.txt</code>
<code>stat</code>	Shows detailed metadata (including inode) for a file	<code>stat /etc/hostname</code>
<code>swapon</code>	Activates a swap file or partition	<code>sudo swapon /swapfile</code>
<code>swapoff</code>	Deactivates a swap file or partition	<code>sudo swapoff /swapfile</code>

Command	What It Does	Common Usage
mkswap	Initializes a file/partition as swap	sudo mkswap /swapfile
fallocate	Quickly allocates a file of a given size	sudo fallocate -l 2G /swapfile

10.15 Chapter Summary

- ✓ Linux names disks and partitions predictably (/dev/sda, /dev/nvme0n1p1) based on the storage controller type.
- ✓ lsblk gives a fast, unprivileged overview; fdisk -l shows the exact partition table.
- ✓ fdisk creates and modifies partitions interactively; nothing is written until you press w.
- ✓ mkfs.ext4, mkfs.xfs, and friends format a partition, permanently erasing whatever was there.
- ✓ mount attaches a filesystem to a directory; /etc/fstab makes that attachment survive a reboot, ideally keyed by UUID.
- ✓ df -h and df -i track space and inode usage; du -sh finds exactly which directories are consuming it.
- ✓ LVM (pvcreate, vgcreate, lvcreate, lvextend) lets you grow storage online without unmounting.
- ✓ Symbolic links point to a path and can break or cross filesystems; hard links point to the same inode and can't.
- ✓ Every file's metadata lives in its inode — but not its filename, which lives in the directory entry.
- ✓ Swap space, whether a partition or a simple file, gives the kernel breathing room when RAM runs low.

What's Coming in Chapter 11: System Logs, Services & systemd

- What systemd is and why it replaced the old init system
- Controlling services with systemctl: start, stop, restart, enable, disable, status
- Reading logs the modern way with journalctl, filtered by time, unit, and priority
- The structure of /var/log and the key log files every admin checks first
- Keeping logs under control with logrotate
- Writing your own systemd service unit file

- Scheduling recurring tasks with cron and crontab
- One-time scheduling with at, and catching up missed jobs with anacron

Every command you've run so far has been reactive. Chapter 11 is where Linux starts working for you in the background.

Author

Ashik A

github.com/ashihh07

This guide was created, organized, refined, and designed by Ashik A using GitHub documentation, industry best practices, publicly available learning resources, and AI-assisted research.
